

LAPORAN TUGAS BESAR

STRATEGI ALGORITMA

Pemanfaatan Algoritma Greedy dalam Pembuatan Bot Permainan Robocode Tank Royale

Nama Kelompok: terserah (RB)

Anggota:

- Natan Handani (124140175)
- Arbani Hafizh Saragih (124140139)
- Rendi Putra (124140030)



Program Studi Teknik Informatika

Institut Teknologi Sumatera

Tahun 2026

BAB I DESKRIPSI TUGAS	3
BAB II LANDASAN TEORI	5
2.1 Algoritma	6
2.2 Algoritma Greedy	6
2.3 Elemen Algoritma Greedy	7
2.4 Heuristic dalam Algoritma Greedy	8
2.5 Robocode Tank Royale	8
BAB III APLIKASI STRATEGI GREEDY	9
3.1 Mapping Persoalan RoboCode Tank Royale Ke Elemen Algoritma Greedy	9
3.2 Eksplorasi Alternatif Strategi Greedy	9
3.2.1 Greedy by Closest	9
3.2.2 Greedy by Energy	9
3.2.3 Greedy by Survival	9
3.2.4 Greedy by Score	9
3.3 Analisis Efisiensi dan Efektivitas Alternatif Solusi	9
3.4 Strategi greedy yang dipilih	9
BAB IV IMPLEMENTASI DAN PENGUJIAN	9
4.1 Implementasi Alternatif Solusi Greedy	9
4.1.1 Implementasi greedy Closest Bot	9
4.1.2 Implementasi Greedy Energy Bot	9
4.1.3 Implementasi Greedy Survival Bot	9
4.1.4 Implementasi Greedy Main Bot	9
4.2 Struktur Data, Fungsi, dan Prosedur Program	9
4.3 Pengujian Program	9
BAB V KESIMPULAN DAN SARAN	9
5.1 Kesimpulan	9
5.2 Saran	9
Lampiran	9
DAFTAR PUSTAKA	9

DESKRIPSI TUGAS

Robocode adalah permainan pemrograman yang bertujuan untuk membuat kode bot dalam bentuk tank virtual untuk berkompetisi melawan bot lain di arena. Pertempuran Robocode berlangsung hingga bot-bot bertarung hanya tersisa satu seperti permainan Battle Royale, 1 IF25--21013 Strategi Algoritma – Tugas Besar karena itulah permainan ini dinamakan Tank Royale. Nama Robocode adalah singkatan dari "Robot code," yang berasal dari versi asli/pertama permainan ini. Robocode Tank Royale adalah evolusi/versi berikutnya dari permainan ini, di mana bot dapat berpartisipasi melalui Internet/jaringan. Dalam permainan ini, pemain berperan sebagai programmer bot dan tidak memiliki kendali langsung atas permainan. Pemain hanya bertugas untuk membuat program yang menentukan logika atau "otak" bot. Program yang dibuat akan berisi instruksi tentang cara bot bergerak, mendeteksi bot lawan, menembakkan senjatanya, serta bagaimana bot bereaksi terhadap berbagai kejadian selama pertempuran. Pada Tugas Besar pertama Strategi Algoritma ini, mahasiswa diminta untuk membuat sebuah bot yang nantinya akan dipertandingkan satu sama lain. Tentunya mahasiswa harus menggunakan strategi greedy dalam membuat bot ini..

BAB II

LANDASAN TEORI

2.1 Algoritma

Algoritma merupakan urutan langkah-langkah logis dan sistematis yang digunakan untuk menyelesaikan suatu permasalahan. Dalam bidang pemrograman, algoritma menjadi dasar dalam penyusunan instruksi agar komputer dapat melakukan proses tertentu sesuai tujuan yang diinginkan. Setiap algoritma harus memiliki masukan, proses, dan keluaran yang jelas sehingga solusi yang dihasilkan dapat diuji dan dievaluasi.

Dalam penyelesaian masalah komputasi, pemilihan algoritma sangat berpengaruh terhadap hasil program. Algoritma yang baik tidak hanya mampu menghasilkan solusi, tetapi juga perlu mempertimbangkan efisiensi waktu, penggunaan sumber daya, serta kesesuaian dengan karakteristik masalah. Pada permainan seperti Robocode Tank Royale, algoritma digunakan untuk menentukan perilaku bot, seperti bergerak, mencari musuh, memilih target, menembak, dan menghindari ancaman.

2.2 Algoritma Greedy

Algoritma greedy adalah salah satu strategi penyelesaian masalah yang mengambil keputusan terbaik pada setiap langkah berdasarkan kondisi saat itu. Pendekatan ini memilih solusi yang tampak paling menguntungkan secara lokal dengan harapan

dapat menghasilkan solusi akhir yang baik secara keseluruhan. MIT OpenCourseWare menjelaskan bahwa algoritma greedy membuat pilihan terbaik secara lokal tanpa mempertimbangkan seluruh kemungkinan kondisi di masa depan.

Ciri utama dari algoritma greedy adalah keputusan diambil secara bertahap dan tidak selalu ditinjau kembali setelah dipilih. Hal ini membuat greedy relatif sederhana, cepat, dan mudah diterapkan. Namun, algoritma greedy tidak selalu menjamin solusi paling optimal secara global, terutama pada permasalahan yang kondisinya berubah-ubah atau memiliki banyak kemungkinan keputusan. Oleh karena itu, greedy lebih tepat digunakan ketika masalah dapat disederhanakan menjadi serangkaian pilihan lokal yang masuk akal.

Dalam konteks Robocode Tank Royale, algoritma greedy dapat digunakan karena bot harus mengambil keputusan secara cepat selama pertandingan berlangsung. Bot tidak memiliki cukup waktu untuk menghitung seluruh kemungkinan gerakan musuh pada masa depan. Oleh sebab itu, bot dapat memilih aksi terbaik berdasarkan informasi yang tersedia saat itu, misalnya memilih musuh terdekat, musuh dengan energi paling rendah, atau target dengan peluang serangan paling besar.

2.3 Elemen Algoritma Greedy

Penerapan algoritma greedy umumnya terdiri dari beberapa elemen utama, yaitu himpunan kandidat, himpunan solusi, fungsi seleksi, fungsi kelayakan, dan fungsi objektif. Elemen-elemen ini digunakan untuk membantu algoritma menentukan pilihan terbaik pada setiap tahap pengambilan keputusan.

Himpunan kandidat adalah kumpulan pilihan yang tersedia untuk dipertimbangkan. Dalam Robocode Tank Royale, kandidat dapat berupa seluruh bot lawan yang berhasil terdeteksi oleh radar. Setiap kandidat memiliki atribut tertentu, seperti posisi, jarak, energi, dan tingkat ancaman terhadap bot utama.

Himpunan solusi adalah pilihan yang akhirnya diambil dari himpunan kandidat. Pada permainan Robocode Tank Royale, solusi dapat berupa satu target yang dipilih untuk diserang atau dihindari. Target tersebut dipilih berdasarkan aturan atau strategi greedy yang digunakan oleh bot.

Fungsi seleksi adalah aturan yang digunakan untuk memilih kandidat terbaik. Fungsi ini menjadi inti dari algoritma greedy karena menentukan keputusan yang akan diambil pada setiap kondisi permainan. Contohnya, bot dapat memilih musuh dengan jarak paling dekat, energi paling rendah, atau skor peluang tertinggi.

Fungsi kelayakan digunakan untuk memastikan bahwa keputusan yang dipilih dapat dilakukan sesuai batasan permainan. Misalnya, bot hanya dapat menembak ketika senjata sudah siap dan tidak dalam kondisi panas. Selain itu, bot juga harus mempertimbangkan posisi terhadap dinding arena agar tidak mudah terjebak.

Fungsi objektif adalah tujuan utama yang ingin dicapai oleh algoritma. Dalam Robocode Tank Royale, tujuan utama bot adalah memperoleh skor setinggi mungkin dengan cara memberikan damage kepada musuh, bertahan hidup lebih lama, dan memenangkan ronde pertandingan.

2.4 Heuristic dalam Algoritma Greedy

Heuristic merupakan pendekatan praktis yang digunakan untuk membantu pengambilan keputusan ketika solusi terbaik sulit dihitung secara lengkap. Dalam algoritma greedy, heuristic digunakan sebagai dasar pemilihan kandidat yang dianggap paling menguntungkan pada kondisi tertentu. Heuristic tidak selalu menghasilkan solusi paling optimal, tetapi dapat memberikan keputusan yang cepat dan cukup efektif.

Pada Robocode Tank Royale, heuristic dapat berupa aturan sederhana seperti memilih musuh terdekat, memilih musuh dengan energi terendah, atau memilih musuh yang berada dalam jarak tembak efektif. Pendekatan ini sesuai dengan karakter permainan

yang dinamis karena bot harus terus merespons perubahan posisi musuh, arah peluru, dan kondisi arena.

Penggunaan heuristic juga memungkinkan bot memiliki strategi yang berbeda-beda. Bot yang menggunakan heuristic jarak akan lebih agresif terhadap musuh terdekat. Bot yang menggunakan heuristic energi akan lebih fokus pada peluang eliminasi. Sementara itu, bot yang menggunakan gabungan beberapa heuristic dapat mempertimbangkan lebih banyak aspek sebelum memilih target.

2.5 Robocode Tank Royale

Robocode Tank Royale adalah permainan pemrograman di mana pemain membuat program untuk mengendalikan bot berbentuk tank virtual yang bertarung di dalam arena. Pemain tidak mengendalikan bot secara langsung, tetapi menulis program yang mengatur cara bot bergerak, memindai musuh, menembak, dan merespons kejadian selama pertandingan. Dokumentasi resmi Robocode Tank Royale menjelaskan bahwa logika bot sepenuhnya berada di dalam program yang dibuat oleh pemain.

Tujuan utama dalam Robocode Tank Royale adalah membuat bot mampu bertahan hidup selama mungkin dan mengalahkan bot lawan. Bot akan memperoleh skor lebih baik jika mampu memberikan damage kepada musuh dan bertahan lebih lama dalam pertempuran. Dengan demikian, strategi bot tidak hanya harus berfokus pada serangan, tetapi juga harus memperhatikan aspek pertahanan dan kemampuan menghindari serangan lawan.

Permainan ini sesuai digunakan sebagai media penerapan algoritma karena setiap bot harus mengambil keputusan secara otomatis berdasarkan kondisi arena. Keputusan tersebut meliputi arah pergerakan, pemilihan target, waktu menembak, dan respons terhadap serangan musuh. Dalam mode melee atau pertandingan dengan lebih dari dua bot, strategi bertahan hidup dan pemilihan ancaman menjadi semakin penting karena bot harus menghadapi beberapa lawan sekaligus.

BAB III

APLIKASI STRATEGI GREEDY

3.1 Mapping Persoalan RoboCode Tank Royale Ke Elemen Algoritma Greedy

Permainan Robocode Tank Royale dapat dipandang sebagai persoalan pengambilan keputusan berulang, karena bot harus terus menentukan aksi terbaik pada setiap kondisi permainan. Setiap ronde menghasilkan kondisi yang berubah-ubah, seperti posisi musuh, jarak musuh, energi musuh, arah peluru, serta posisi bot terhadap dinding arena. Dalam konteks ini, algoritma greedy digunakan untuk mengambil keputusan terbaik berdasarkan informasi lokal yang tersedia saat itu, tanpa menghitung seluruh kemungkinan kondisi permainan pada masa depan.

Elemen pertama dalam pemetaan algoritma greedy adalah **himpunan kandidat**. Dalam Robocode Tank Royale, kandidat dapat berupa seluruh bot lawan yang berhasil terdeteksi radar. Setiap bot lawan memiliki atribut yang dapat dipertimbangkan dalam pengambilan keputusan, seperti koordinat posisi, jarak terhadap bot utama, energi tersisa, dan potensi ancaman. Kandidat ini berubah secara dinamis karena musuh terus bergerak selama pertandingan berlangsung.

Elemen kedua adalah **himpunan solusi**. Solusi dalam konteks ini adalah target yang dipilih oleh bot untuk diserang atau direspons. Dari beberapa kandidat musuh yang terdeteksi, bot hanya memilih satu target terbaik pada satu siklus keputusan. Target tersebut kemudian digunakan sebagai dasar untuk mengarahkan senjata, menembak, atau menentukan arah pergerakan.

Elemen ketiga adalah **fungsi seleksi**. Fungsi seleksi merupakan bagian utama dari strategi greedy, karena menentukan kandidat mana yang dianggap paling optimal pada kondisi saat itu. Dalam tugas ini, fungsi seleksi dapat dirancang berdasarkan beberapa pendekatan, misalnya memilih musuh dengan jarak terdekat, memilih musuh dengan energi terendah, memilih musuh yang paling mengancam keselamatan, atau memilih target berdasarkan skor gabungan dari beberapa parameter.

Elemen keempat adalah **fungsi kelayakan**. Fungsi kelayakan digunakan untuk memastikan bahwa keputusan yang diambil masih dapat dilakukan oleh bot dalam aturan permainan. Misalnya, bot hanya menembak ketika $\text{GunHeat} == 0$, karena senjata tidak dapat ditembakkan terus-menerus tanpa jeda pendinginan. Selain itu, bot juga harus mempertimbangkan posisi terhadap dinding dan jarak musuh agar tidak mudah terjebak atau mengalami collision damage secara berulang.

Elemen kelima adalah **fungsi objektif**. Tujuan utama bot adalah memperoleh skor setinggi mungkin dalam pertandingan. Skor dapat diperoleh melalui damage peluru, survival, bonus survival, dan kemenangan ronde. Oleh karena itu, strategi greedy

yang baik tidak hanya mengejar musuh secara agresif, tetapi juga harus mempertimbangkan peluang tembakan mengenai target, peluang eliminasi musuh, dan kemampuan bot untuk bertahan hidup selama pertandingan.

Elemen Greedy	Implementasi pada Robocode Tank Royale
Himpunan kandidat	Seluruh bot lawan yang terdeteksi radar
Himpunan solusi	Target yang dipilih untuk diserang atau dihindari
Fungsi seleksi	Pemilihan target berdasarkan jarak, energi, risiko, atau skor gabungan
Fungsi kelayakan	Target dapat ditembak, senjata siap, bot tidak dalam posisi berisiko tinggi
Fungsi objektif	Memaksimalkan skor pertandingan melalui damage, survival, dan eliminasi lawan

3.2 Eksplorasi Alternatif Strategi Greedy

3.2.1 Greedy by Closest

Strategi **Greedy by Closest** menggunakan jarak sebagai dasar utama pemilihan target. Pada strategi ini, bot memilih musuh yang posisinya paling dekat dari posisi bot saat ini. Dasar pemikirannya adalah semakin dekat jarak musuh, semakin besar peluang peluru mengenai target karena waktu tempuh peluru lebih pendek dan perubahan posisi musuh relatif lebih mudah direspons.

Strategi ini diimplementasikan pada GreedyClosestBot. Setiap kali radar mendeteksi musuh, bot menghitung jarak musuh menggunakan fungsi $\text{DistanceTo}(e.X, e.Y)$. Jika

jarak musuh yang baru terdeteksi lebih kecil daripada jarak target sebelumnya, maka musuh tersebut dijadikan target baru. Setelah target dipilih, bot mengarahkan senjata ke koordinat target menggunakan `GunBearingTo(targetX, targetY)` dan menembak dengan fire power tetap.

Heuristic yang digunakan:

- Memilih musuh jarak terdekat dari posisi bot saat ini

3.2.2 Greedy by Energy

Strategi **Greedy by Lowest Energy** memilih musuh berdasarkan energi paling rendah. Pada strategi ini, bot memprioritaskan musuh yang sudah lemah karena peluang untuk mengeliminasi musuh tersebut lebih besar. Pendekatan ini berorientasi pada efisiensi serangan, karena bot tidak selalu menyerang musuh yang paling dekat, tetapi memilih target yang paling mudah dikalahkan.

Strategi ini diimplementasikan pada `GreedyEnergyBot`. Saat radar mendeteksi bot lawan, energi musuh dibandingkan dengan energi target terbaik sebelumnya. Jika energi musuh lebih rendah, maka musuh tersebut dipilih sebagai target. Apabila terdapat dua musuh dengan energi yang sama, bot memilih musuh yang jaraknya lebih dekat sebagai pertimbangan tambahan.

Heuristic yang digunakan:

- Memilih musuh energi terendah
- Jika ada yang sama, memilih yang terdekat

3.2.3 Greedy by Survival

Strategi **Greedy by Survival Risk** berfokus pada upaya mempertahankan hidup selama pertandingan. Berbeda dari dua strategi sebelumnya yang lebih menyerang, strategi ini menilai musuh terdekat sebagai ancaman utama. Musuh yang dekat

dianggap lebih berbahaya karena memiliki peluang lebih besar untuk menembak, menabrak, atau memaksa bot berada dalam posisi sempit.

Strategi ini diimplementasikan pada GreedySurvivalBot. Bot menggunakan radar untuk mendeteksi musuh, kemudian memilih musuh dengan jarak paling dekat sebagai sumber ancaman. Jika musuh terlalu dekat, bot akan menjauh. Jika musuh berada pada jarak sedang, bot bergerak menyamping agar tidak mudah ditembak lurus. Jika belum ada ancaman yang terdeteksi, bot melakukan gerakan patroli sederhana.

Heuristic yang digunakan:

- Memilih musuh terdekat sebagai ancaman
- Melakukan gerakan untuk menjaga jarak

3.2.4 Greedy by Score

Strategi **Greedy by Opportunity Score** menggunakan pendekatan multi-kriteria dalam memilih target. Bot tidak hanya mempertimbangkan satu parameter, tetapi menggabungkan beberapa faktor yang dianggap relevan terhadap peluang memperoleh skor. Dalam strategi ini, target dievaluasi berdasarkan jarak musuh, energi musuh, dan kelayakan jarak tembak. Target dengan skor tertinggi dipilih sebagai keputusan greedy terbaik pada kondisi saat itu.

Strategi ini diimplementasikan pada GreedyMainBot. Setiap musuh yang terdeteksi radar diberi skor menggunakan fungsi evaluasi sebagai berikut:

$$\text{Score}(\text{target}) = 0.45 \times \text{DistanceScore} + 0.35 \times \text{EnergyScore} + 0.20 \times \text{RangeScore}$$

DistanceScore memberi nilai lebih tinggi kepada musuh yang lebih dekat karena peluang peluru mengenai target lebih besar. EnergyScore memberi nilai lebih tinggi kepada musuh yang energinya lebih rendah karena peluang eliminasi lebih besar.

RangeScore memberi nilai tambahan kepada musuh yang berada pada jarak tembak efektif. Setelah seluruh kandidat dievaluasi, bot memilih target dengan nilai skor tertinggi.

Heuristic yang digunakan:

- Memilih target skor tinggi berdasarkan jangkauan, jarak, energy

3.3 Analisis Efisiensi dan Efektivitas Alternatif Solusi

Keempat alternatif strategi greedy memiliki karakteristik yang berbeda. GreedyClosestBot merupakan strategi paling sederhana karena hanya menggunakan jarak sebagai dasar seleksi. Strategi ini efisien dari sisi komputasi karena hanya membutuhkan perhitungan jarak, tetapi efektivitasnya bergantung pada kondisi arena. Jika target terdekat berada dalam posisi yang menguntungkan, strategi ini dapat menghasilkan serangan yang cepat. Namun, jika target terdekat memiliki energi tinggi atau berada dalam posisi yang sulit ditembak, strategi ini dapat menjadi kurang optimal.

GreedyEnergyBot lebih berorientasi pada peluang eliminasi. Dengan menyerang musuh berenergi rendah, bot berusaha memperoleh keuntungan dari kondisi lawan yang sudah lemah. Strategi ini efektif jika target berenergi rendah masih berada dalam jarak yang dapat ditembak. Akan tetapi, jika target terlalu jauh, strategi ini dapat menurunkan efisiensi karena bot tetap memilih target berdasarkan energi, bukan peluang tembakan mengenai sasaran.

GreedySurvivalBot lebih unggul dalam aspek bertahan hidup. Strategi ini relevan karena survival merupakan salah satu komponen penting dalam perolehan skor Robocode. Bot yang mampu bertahan lebih lama memiliki peluang lebih besar memperoleh survival score. Namun, strategi ini tidak selalu menghasilkan damage

tinggi karena fokus utamanya adalah mengurangi risiko, bukan menyerang secara agresif.

GreedyMainBot memiliki efektivitas paling seimbang karena menggunakan skor gabungan. Dengan mengombinasikan jarak, energi, dan kelayakan jarak tembak, bot dapat memilih target yang tidak hanya dekat, tetapi juga memiliki peluang serangan yang lebih baik. Dari sisi efisiensi, strategi ini masih cukup ringan karena perhitungan skor hanya dilakukan pada setiap musuh yang terdeteksi radar. Oleh karena itu, strategi ini dipandang paling layak untuk digunakan sebagai bot utama.

Alternatif	Dasar Greedy	Keunggulan	Kelemahan
Greedy by Distance	Jarak terdekat	Sederhana dan cepat	Bisa menyerang target dekat yang berbahaya
Greedy by Lowest Energy	Energi terendah	Berpeluang mengeliminasi musuh lemah	Target energi rendah bisa terlalu jauh
Greedy by Survival Risk	Ancaman terdekat	Meningkatkan peluang bertahan hidup	Kurang agresif dalam menghasilkan damage
Greedy by Opportunity Score	Skor gabungan	Lebih seimbang dan adaptif	Bergantung pada bobot heuristic

3.4 Strategi greedy yang dipilih

Strategi greedy yang dipilih untuk bot utama adalah **Greedy by Opportunity Score**. Strategi ini dipilih karena mampu mengevaluasi target secara lebih menyeluruh dibandingkan strategi tunggal. Dalam permainan Robocode Tank Royale, memilih target hanya berdasarkan jarak belum tentu optimal karena musuh terdekat dapat

memiliki energi tinggi dan berisiko menyebabkan tabrakan. Sebaliknya, memilih target hanya berdasarkan energi juga belum tentu optimal karena musuh berenergi rendah mungkin berada terlalu jauh sehingga sulit ditembak. Oleh karena itu, pendekatan skor gabungan dianggap lebih sesuai untuk menghadapi kondisi permainan yang dinamis.

Pada strategi ini, setiap musuh yang terdeteksi radar akan dihitung skornya berdasarkan tiga komponen utama. Komponen pertama adalah DistanceScore, yaitu skor yang meningkat ketika jarak musuh semakin dekat. Komponen kedua adalah EnergyScore, yaitu skor yang meningkat ketika energi musuh semakin rendah. Komponen ketiga adalah RangeScore, yaitu skor yang menilai apakah musuh berada dalam jarak tembak yang efektif. Ketiga komponen tersebut kemudian digabungkan menggunakan bobot tertentu untuk menghasilkan skor peluang target.

Bobot terbesar diberikan pada DistanceScore karena jarak sangat memengaruhi peluang tembakan mengenai target. Bobot kedua diberikan pada EnergyScore karena musuh berenergi rendah memiliki peluang lebih besar untuk dieliminasi. Bobot terakhir diberikan pada RangeScore untuk memastikan bahwa target yang dipilih tetap berada pada jarak serang yang layak. Dengan demikian, target yang dipilih bukan hanya musuh yang dekat, tetapi musuh yang memberikan peluang serangan paling menguntungkan pada kondisi saat itu.

BAB IV

IMPLEMENTASI DAN PENGUJIAN

4.1 Implementasi Alternatif Solusi Greedy

Implementasi program dilakukan dengan menggunakan bahasa pemrograman C# dan Robocode Tank Royale Bot API. Setiap bot dibuat sebagai class turunan dari Bot, kemudian dijalankan melalui method Main() dan konfigurasi bot dibaca melalui file .json masing-masing. Secara umum, setiap bot memiliki pola dasar yang sama, yaitu melakukan scan terhadap musuh, memilih target berdasarkan strategi greedy tertentu, mengarahkan senjata, menembak ketika GunHeat == 0, serta melakukan pergerakan untuk menghindari posisi diam. Perbedaan utama antarbot terletak pada fungsi seleksi target yang digunakan.

4.1.1 Implementasi greedy Closest Bot

Pseudocode GreedyClosestBot;

Mulai program GreedyClosestBot

Inisialisasi:

nearestDistance = nilai maksimum

targetFound = false

nearestX, nearestY = 0

Selama bot berjalan:

nearestDistance = nilai maksimum

targetFound = false

Putar radar 360 derajat

Jika radar mendeteksi musuh:

Hitung jarak musuh dari posisi bot

Jika jarak musuh lebih kecil dari nearestDistance:

Simpan jarak musuh sebagai nearestDistance

Simpan koordinat musuh sebagai nearestX dan nearestY

targetFound = true

Jika targetFound bernilai true:

Arahkan senjata ke koordinat target

Jika GunHeat == 0:

Tembak dengan fire power 1

Lakukan gerakan maju, belok, mundur, dan belok kembali

Jika terkena peluru:

Hitung arah datang peluru

Bergerak menjauh dari arah peluru

Jika menabrak dinding:

Mundur dan berbelok

Jika menabrak bot lain:

Mundur dan mengubah arah

4.1.2 Implementasi Greedy Energy Bot

Pseudocode GreedyEnergyBot:

Mulai program GreedyEnergyBot

Inisialisasi:

lowestEnergy = nilai maksimum

targetDistance = nilai maksimum

targetFound = false

targetX, targetY = 0

Selama bot berjalan:

lowestEnergy = nilai maksimum

targetDistance = nilai maksimum

targetFound = false

Putar radar 360 derajat

Jika radar mendeteksi musuh:

Ambil nilai energi musuh

Hitung jarak musuh dari posisi bot

Jika energi musuh lebih kecil dari lowestEnergy:

Pilih musuh tersebut sebagai target

Jika energi musuh sama dengan lowestEnergy:

Pilih musuh yang jaraknya lebih dekat

Simpan koordinat, energi, dan jarak target

Jika target ditemukan:

Arahkan senjata ke target

Jika GunHeat == 0:

Tembak dengan fire power 1

Lakukan gerakan dasar agar bot tidak diam

Jika terkena peluru:

Bergerak menjauh dari arah peluru

Jika menabrak dinding:

Mundur dan berbelok

Jika menabrak bot:

Mundur dan mengubah arah

4.1.3 Implementasi Greedy Survival Bot

Pseudocode GreedySurvivalBot:

Mulai program GreedySurvivalBot

Inisialisasi:

threatDistance = nilai maksimum

threatFound = false

threatX, threatY = 0

Selama bot berjalan:

threatDistance = nilai maksimum

threatFound = false

Putar radar 360 derajat

Jika radar mendeteksi musuh:

Hitung jarak musuh

Jika jarak musuh lebih kecil dari threatDistance:

Simpan musuh sebagai ancaman utama

Simpan koordinat musuh

threatFound = true

Jika threatFound bernilai true:

Jika ancaman terlalu dekat:

Bot menjauh dari ancaman

Jika ancaman berada pada jarak sedang:

Bot bergerak menyamping

Jika ancaman jauh:

Bot bergerak normal

Arahkan senjata ke ancaman

Jika GunHeat == 0:

Tembak dengan fire power 1

Jika tidak ada ancaman:

Lakukan patroli

Jika terkena peluru:

Menghindar tegak lurus dari arah peluru

Jika menabrak dinding:

Mundur dan berbelok

Jika menabrak bot:

Mundur dan mengubah arah

4.1.4 Implementasi Greedy Main Bot

Pseudocode GreedyMainBot:

Mulai program GreedyMainBot

Inisialisasi:

bestScore = nilai minimum

targetDistance = nilai maksimum

targetEnergy = 100

targetFound = false

targetX, targetY = 0

Atur agar gun dan radar lebih independen dari gerakan body:

AdjustGunForBodyTurn = true

AdjustRadarForBodyTurn = true

AdjustRadarForGunTurn = true

Selama bot berjalan:

bestScore = nilai minimum

targetFound = false

Putar radar 360 derajat

Jika radar mendeteksi musuh:

Hitung jarak musuh

Hitung DistanceScore

Hitung EnergyScore

Hitung RangeScore

Hitung totalScore:

$$\text{totalScore} = 0.45 \times \text{DistanceScore}$$

$$+ 0.35 \times \text{EnergyScore}$$

$$+ 0.20 \times \text{RangeScore}$$

Jika totalScore lebih besar dari bestScore:

Simpan musuh sebagai target terbaik

Simpan koordinat, jarak, energi, dan skor target

Jika target ditemukan:

Arahkan senjata ke target

Jika GunHeat == 0:

 Tembak dengan fire power 1

Jika target terlalu dekat:

 Bot menjauh

Jika target berada pada jarak sedang:

 Bot bergerak menyamping

Jika target jauh:

 Bot mendekat secara bertahap

Jika target tidak ditemukan:

 Lakukan gerakan pencarian

Jika terkena peluru:

 Bergerak tegak lurus dari arah peluru

Jika menabrak dinding:

 Mundur dan berbelok

Jika menabrak bot:

Mundur dan mengubah arah

4.2 Struktur Data, Fungsi, dan Prosedur Program

Implementasi bot menggunakan beberapa struktur data sederhana berupa variabel numerik dan boolean. Struktur data ini digunakan untuk menyimpan informasi target yang diperoleh dari hasil scan radar. Pada GreedyClosestBot, variabel utama yang digunakan adalah nearestDistance, nearestX, nearestY, dan targetFound. Variabel tersebut digunakan untuk menyimpan jarak dan koordinat musuh terdekat.

Pada GreedyEnergyBot, variabel utama yang digunakan adalah lowestEnergy, targetDistance, targetX, targetY, dan targetFound. Variabel lowestEnergy berfungsi untuk menyimpan energi musuh paling rendah yang terdeteksi, sedangkan targetDistance digunakan sebagai pembanding jika terdapat lebih dari satu musuh dengan nilai energi yang sama.

Pada GreedySurvivalBot, variabel utama yang digunakan adalah threatDistance, threatX, threatY, dan threatFound. Variabel ini digunakan untuk menentukan musuh yang menjadi ancaman terbesar berdasarkan jarak. Pada GreedyMainBot, variabel utama yang digunakan adalah bestScore, targetDistance, targetEnergy, targetX, targetY, dan targetFound. Variabel bestScore menjadi bagian penting karena menyimpan nilai evaluasi tertinggi dari target yang terdeteksi.

Beberapa fungsi dan prosedur utama yang digunakan pada implementasi bot dapat dijelaskan sebagai berikut.

Fungsi/Prosedur	Kegunaan
Run()	Mengatur warna bot, menjalankan loop utama,

	melakukan scan, serta memanggil fungsi gerak dan serang
OnScannedBot()	Event yang dijalankan ketika radar mendeteksi bot lawan
DistanceTo(x, y)	Menghitung jarak bot terhadap koordinat musuh
GunBearingTo(x, y)	Menghitung arah putaran senjata menuju target
BearingTo(x, y)	Menghitung arah relatif bot terhadap posisi target
Fire(1)	Menembak target dengan fire power 1
GunHeat	Mengecek apakah senjata sudah siap menembak
OnHitByBullet()	Event ketika bot terkena peluru
OnHitWall()	Event ketika bot menabrak dinding
OnHitBot()	Event ketika bot bertabrakan dengan bot lain
AdjustGunForBodyTurn	Membuat gun tidak sepenuhnya bergantung pada putaran body
AdjustRadarForBodyTurn	Membuat radar tidak sepenuhnya bergantung pada putaran body
AdjustRadarForGunTurn	Membuat radar tidak sepenuhnya bergantung pada putaran gun

4.3 Pengujian Program

Results for 3 rounds

Rank	Name	Total Score	Survival	Surv. Bonus	Bullet Dmg.	Bullet Bonus	Ram Dmg.	Ram Bonus	1sts	2nds	3rds
1	Greedy Closest Bot 1.0	408	200	30	156	11	11	0	2	0	0
2	Greedy Energy Bot 1.0	277	150	0	116	0	11	0	0	1	1
3	Greedy Main Bot 1.0	260	100	0	140	0	20	0	0	1	1
4	Greedy Survival Bot 1.0	85	0	0	72	0	13	0	0	0	0

Results for 3 rounds

Rank	Name	Total Score	Survival	Surv. Bonus	Bullet Dmg.	Bullet Bonus	Ram Dmg.	Ram Bonus	1sts	2nds	3rds
1	Greedy Closest Bot 1.0	368	200	30	120	13	5	0	1	1	0
2	Greedy Energy Bot 1.0	258	150	0	96	0	12	0	0	1	1
3	Greedy Main Bot 1.0	198	50	0	136	0	12	0	1	0	0
4	Greedy Survival Bot 1.0	50	0	0	64	0	10	0	0	0	1

Results for 3 rounds

Rank	Name	Total Score	Survival	Surv. Bonus	Bullet Dmg.	Bullet Bonus	Ram Dmg.	Ram Bonus	1sts	2nds	3rds
1	Greedy Main Bot 1.0	316	150	30	128	0	8	0	1	1	0
2	Greedy Closest Bot 1.0	222	100	0	112	10	0	0	0	1	1
3	Greedy Energy Bot 1.0	164	50	0	108	0	6	0	1	0	1
4	Greedy Survival Bot 1.0	30	0	0	20	0	10	0	0	0	0

Robocode Tank Royale
Build the best - destroy the rest!

Bot	Pengujian 1	Pengujian 2	Pengujian 3	Total	Rata-rata
GreedyMainBot	316	260	198	774	258,00
GreedyClosestBot	222	408	368	998	332,67
GreedyEnergyBot	164	277	258	699	233,00
GreedySurvivalBot	30	85	124	239	79,67

Dari tabel tersebut, GreedyClosestBot memperoleh total score tertinggi, yaitu 998, dengan rata-rata 332,67. Hasil ini menunjukkan bahwa strategi Greedy by Distance cukup efektif dalam skenario pengujian yang dilakukan. Pemilihan musuh terdekat memberikan keuntungan karena target yang berada pada jarak dekat lebih mudah ditembak dan lebih cepat dijangkau. Selain itu, pada Pengujian 2 dan Pengujian 3, GreedyClosestBot juga memperoleh survival score yang tinggi, sehingga skor totalnya meningkat secara signifikan.

GreedyMainBot, yang menggunakan strategi Greedy by Opportunity Score, memperoleh total score 774 dengan rata-rata 258,00. Pada Pengujian 1, bot ini berhasil menempati peringkat pertama dengan skor 316, tetapi pada Pengujian 2 dan Pengujian 3 posisinya turun. Jika dilihat lebih rinci, kelemahan utama GreedyMainBot bukan terletak pada kemampuan menyerang, melainkan pada aspek survival. Pada Pengujian 3, misalnya, GreedyMainBot menghasilkan bullet damage sebesar 136, lebih tinggi daripada GreedyClosestBot yang menghasilkan bullet damage 120, tetapi skor totalnya tetap lebih rendah karena survival score hanya 50. Hal ini menunjukkan bahwa fungsi pemilihan target pada GreedyMainBot sudah mampu menghasilkan serangan yang cukup efektif, tetapi pola pergerakannya belum cukup stabil untuk mempertahankan bot sampai akhir ronde.

GreedyEnergyBot memperoleh total score 699 dengan rata-rata 233,00. Strategi ini cukup konsisten berada pada peringkat menengah. Pemilihan target berdasarkan energi terendah dapat meningkatkan peluang eliminasi, tetapi strategi ini memiliki keterbatasan karena musuh dengan energi rendah belum tentu berada pada posisi yang mudah ditembak. Jika target berenergi rendah berada terlalu jauh atau berada di luar sudut tembak efektif, bot dapat kehilangan momentum serangan. Dengan demikian, strategi Greedy by Lowest Energy cukup berguna sebagai pendekatan ofensif, tetapi kurang optimal jika tidak dikombinasikan dengan pertimbangan jarak.

GreedySurvivalBot memperoleh total score paling rendah, yaitu 239, dengan rata-rata 79,67. Hasil ini menunjukkan bahwa strategi bertahan saja belum cukup untuk memperoleh skor tinggi. Meskipun secara konsep bot ini dirancang untuk menghindari ancaman dan menjaga jarak aman, implementasi movement yang digunakan belum mampu meningkatkan survival secara konsisten. Selain itu, karena bot ini tidak terlalu agresif dalam menyerang, bullet damage yang dihasilkan relatif lebih rendah dibandingkan bot lain. Akibatnya, skor totalnya tertinggal.

Jika dilihat dari efektivitas strategi, hasil pengujian menunjukkan bahwa strategi greedy sederhana berbasis jarak justru memberikan hasil paling baik dalam kondisi uji ini. Hal tersebut dapat terjadi karena arena dan jumlah ronde yang digunakan membuat target terdekat menjadi pilihan yang cukup menguntungkan. Musuh yang dekat lebih mudah terkena tembakan, lebih cepat masuk dalam jangkauan serang, dan dapat memberikan damage lebih cepat. Namun, hasil ini tidak berarti bahwa strategi greedy gabungan gagal. GreedyMainBot tetap menunjukkan performa menyerang yang cukup kuat, terutama dari nilai bullet damage yang relatif tinggi.

Dengan demikian, strategi Greedy by Opportunity Score pada GreedyMainBot dapat dikatakan efektif dalam menghasilkan damage, tetapi belum optimal dalam menjaga survival. Kondisi ini menunjukkan bahwa fungsi objektif bot masih lebih dominan pada aspek serangan daripada pertahanan. Untuk meningkatkan performa, strategi greedy gabungan dapat dikembangkan dengan menambahkan komponen risiko, misalnya jarak terhadap dinding, jumlah musuh di sekitar bot, atau frekuensi bot terkena peluru. Dengan penambahan faktor tersebut, bot tidak hanya memilih target yang menguntungkan untuk ditembak, tetapi juga mempertimbangkan keamanan posisi.

Kesimpulannya, berdasarkan tiga kali pengujian, GreedyClosestBot menjadi strategi dengan performa terbaik secara empiris, sedangkan GreedyMainBot tetap menjadi strategi utama yang lebih konseptual karena menggunakan evaluasi multi-kriteria. Hasil ini memperlihatkan karakteristik penting algoritma greedy: keputusan terbaik secara lokal tidak selalu menghasilkan hasil global yang paling optimal. Dalam kasus ini, strategi gabungan mampu menyerang dengan baik, tetapi performa akhirnya tetap dipengaruhi oleh pola gerak, posisi awal, dan kemampuan bertahan selama pertandingan.

BAB V

KESIMPULAN DAN SARAN

5.1 Kesimpulan

Berdasarkan proses perancangan, implementasi, dan pengujian yang telah dilakukan, strategi greedy dapat diterapkan pada Robocode Tank Royale karena permainan ini menuntut bot untuk mengambil keputusan secara cepat berdasarkan kondisi saat ini. Setiap bot lawan yang terdeteksi radar dapat dipandang sebagai himpunan kandidat, sedangkan target yang dipilih menjadi solusi sementara. Fungsi seleksi greedy kemudian digunakan untuk menentukan kandidat terbaik berdasarkan parameter tertentu, seperti jarak, energi musuh, risiko ancaman, atau skor gabungan. Dengan demikian, persoalan Robocode Tank Royale dapat dimodelkan sebagai persoalan pengambilan keputusan lokal yang sesuai dengan karakteristik algoritma greedy.

Empat alternatif strategi greedy telah berhasil diimplementasikan, yaitu GreedyClosestBot, GreedyEnergyBot, GreedySurvivalBot, dan GreedyMainBot. GreedyClosestBot menggunakan pendekatan **Greedy by Distance** dengan memilih musuh terdekat sebagai target. GreedyEnergyBot menggunakan pendekatan **Greedy by Lowest Energy** dengan memilih musuh yang memiliki energi paling rendah. GreedySurvivalBot menggunakan pendekatan **Greedy by Survival Risk** dengan menjadikan musuh terdekat sebagai ancaman yang harus dihindari. Sementara itu, GreedyMainBot menggunakan pendekatan **Greedy by Opportunity Score**, yaitu strategi gabungan berdasarkan jarak musuh, energi musuh, dan kelayakan jarak tembak.

Hasil pengujian menunjukkan bahwa strategi greedy memiliki performa yang berbeda tergantung pada kondisi pertandingan. Dari tiga kali pengujian, GreedyClosestBot memperoleh total score tertinggi, yaitu **998** dengan rata-rata **332,67**, sedangkan GreedyMainBot memperoleh total score **774** dengan rata-rata **258,00**. Hasil ini

menunjukkan bahwa strategi sederhana berbasis jarak dapat menjadi sangat efektif dalam kondisi arena dan pola pertandingan tertentu. Musuh yang lebih dekat cenderung lebih mudah ditembak dan lebih cepat masuk ke jangkauan serangan, sehingga strategi ini menghasilkan skor yang relatif tinggi.

Meskipun GreedyMainBot tidak memperoleh skor tertinggi secara keseluruhan, strategi **Greedy by Opportunity Score** tetap memiliki keunggulan konseptual karena mempertimbangkan lebih dari satu parameter. Bot utama mampu menghasilkan bullet damage yang cukup tinggi, terutama pada Pengujian 3 ketika GreedyMainBot memperoleh bullet damage sebesar **136**, lebih tinggi daripada GreedyClosestBot yang memperoleh **120**. Namun, skor total GreedyMainBot masih tertahan karena nilai survival belum stabil. Hal ini menunjukkan bahwa strategi pemilihan target sudah cukup efektif dalam aspek ofensif, tetapi masih perlu diperkuat pada aspek pergerakan dan pertahanan.

Dengan demikian, dapat disimpulkan bahwa algoritma greedy dapat digunakan secara efektif untuk membangun perilaku bot pada Robocode Tank Royale, terutama dalam proses pemilihan target. Namun, hasil pengujian juga menunjukkan bahwa keputusan greedy yang optimal secara lokal tidak selalu menghasilkan hasil global terbaik. Performa akhir bot tetap dipengaruhi oleh faktor lain, seperti posisi awal, pola pergerakan, kemampuan menghindar, akurasi tembakan, dan interaksi dengan bot lain selama pertandingan.

5.2 Saran

Berdasarkan hasil implementasi dan pengujian, pengembangan bot selanjutnya sebaiknya tidak hanya berfokus pada pemilihan target, tetapi juga pada peningkatan pola pergerakan. Pada GreedyMainBot, fungsi seleksi target sudah mampu menghasilkan serangan yang cukup baik, tetapi survival score masih belum stabil. Oleh karena itu, strategi movement dapat dikembangkan dengan pola yang lebih

adaptif, misalnya dengan mempertimbangkan jarak terhadap dinding, arah datang peluru, perubahan posisi musuh, dan kemungkinan bot terjebak dalam area sempit.

Selain itu, fungsi evaluasi pada GreedyMainBot dapat diperluas dengan menambahkan parameter risiko. Saat ini, skor target dihitung berdasarkan jarak, energi musuh, dan kelayakan jarak tembak. Pada pengembangan berikutnya, skor dapat ditambah dengan faktor seperti tingkat bahaya posisi target, jumlah musuh di sekitar bot, serta peluang bot terkena tembakan setelah mendekati target. Dengan demikian, bot tidak hanya memilih target yang mudah diserang, tetapi juga mempertimbangkan keamanan posisi sebelum melakukan serangan.

Pengujian juga sebaiknya dilakukan dengan jumlah ronde yang lebih banyak agar hasil lebih stabil. Dalam laporan ini, pengujian dilakukan sebanyak tiga kali dengan masing-masing 3 rounds. Untuk memperoleh gambaran performa yang lebih kuat, pengujian dapat diperluas menjadi 10 hingga 20 rounds pada setiap skenario. Selain itu, bot sebaiknya diuji melawan bot asisten atau bot dari kelompok lain agar performanya dapat dinilai dalam kondisi kompetisi yang lebih realistis.

Saran terakhir adalah melakukan tuning terhadap bobot pada rumus **Opportunity Score**. Bobot yang digunakan saat ini adalah 0.45 untuk jarak, 0.35 untuk energi, dan 0.20 untuk jarak tembak efektif. Bobot tersebut masih bersifat heuristic, sehingga masih dapat disesuaikan berdasarkan hasil eksperimen. Jika survival menjadi masalah utama, bobot dapat dimodifikasi dengan menambahkan komponen risiko atau mengurangi agresivitas terhadap target yang terlalu dekat. Dengan pengembangan tersebut, strategi greedy gabungan berpotensi menghasilkan performa yang lebih seimbang antara serangan dan pertahanan.

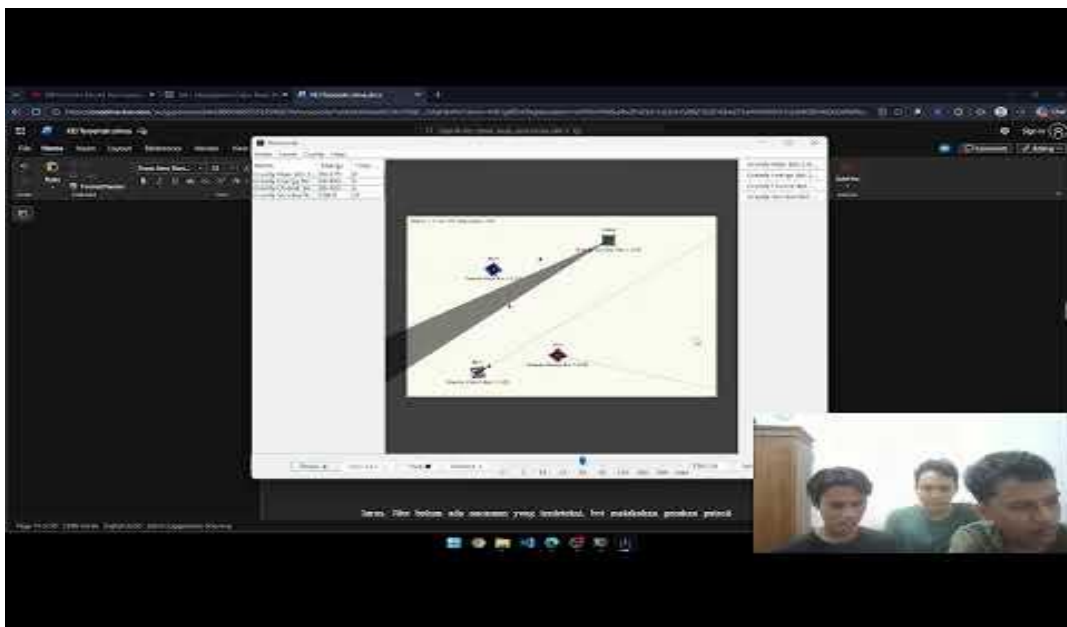
Lampiran

Repository GitHub:

https://github.com/USERNAME_GITHUB/Tubes_terserah

Video demo:

https://youtu.be/7smdtK_uKbk



No	Poin	Ya	Tidak
1	Bot dapat dijalankan pada Engine yang sudah dimodifikasi asisten	✓	
2	Membuat 4 solusi greedy dengan heuristic yang berbeda	✓	
3	Membuat Laporan sesuai dengan spesifikasi	✓	
4	Membuat video bonus dan	✓	

	diunggah pada Youtube		
--	-----------------------	--	--

DAFTAR PUSTAKA

- [1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, Introduction to Algorithms, 4th ed. Cambridge, MA, USA: MIT Press, 2022. [Online]. Available: <https://mitpress.mit.edu/9780262046305/introduction-to-algorithms/>
- [2] E. Demaine, "Lecture 12: Greedy algorithms: Minimum spanning tree," MIT OpenCourseWare, Massachusetts Institute of Technology, 2015. [Online]. Available: <https://ocw.mit.edu/courses/6-046j-design-and-analysis-of-algorithms-spring-2015/resources/lecture-12-greedy-algorithms-minimum-spanning-tree/>
- [3] F. N. Larsen, "Robocode Tank Royale Docs," Robocode Tank Royale, 2022. [Online]. Available: <https://robocode.dev/>. [Accessed: May 23, 2026].
- [4] Robocode Developers, "Robocode Tank Royale," GitHub, 2026. [Online]. Available: <https://github.com/robocode-dev/tank-royale>. [Accessed: May 23, 2026].
- [5] Robocode Tank Royale, "My First Bot for .NET," Robocode Tank Royale Docs, 2022. [Online]. Available: <https://robocode.dev/tutorial/dotnet/my-first-bot-for-dotnet.html>. [Accessed: May 23, 2026].

[6] Robocode Tank Royale, "API Documentation," Robocode Tank Royale Docs, 2022. [Online]. Available: <https://robocode.dev/api/apis.html>. [Accessed: May 23, 2026].

[7] Robocode Tank Royale, "Scoring," Robocode Tank Royale Docs, 2022. [Online]. Available: <https://robocode.dev/articles/scoring.html>. [Accessed: May 23, 2026].

[8] NuGet Gallery, "Robocode.TankRoyale.BotApi," NuGet, 2026. [Online]. Available: <https://www.nuget.org/packages/Robocode.TankRoyale.BotApi>. [Accessed: May 23, 2026].